



UNIVERSIDAD AUTÓNOMA DE BAJA CALIFORNIA
FACULTAD DE CIENCIAS DE LA INGENIERÍA Y TECNOLOGÍA – FCITEC
UNIDAD VALLE DE LAS PALMAS

Ingeniería en Software y Tecnologías Emergentes

Patrones de Software

**Meta 4.2. Investigación documental olores dentro y entre
clases**

Millan Nuñez Jiselle

5to. Semestre

554

Mtra. Abigail Moreno

24 de Abril del 2026

Investigación de olores dentro de las clases:

1. Olores Medibles (Métricas de Código)

Estos se detectan mediante números. Si una clase o método supera ciertos umbrales, empieza a oler mal. Clase grande es una clase que intenta hacer demasiadas cosas violando el principio de responsabilidad única tiene demasiados atributos o líneas de código. Por otra parte, cuanto más largo es un método, más difícil es de entender y mantener. Generalmente, más de 20 a 30 líneas es una señal de alerta.

```
Java
// Clase que hace de todo: maneja datos, cálculos y formato olor a clase grande
public class ProcesadorUsuario {
    private String nombre;
    private String email;
    // ... otros 20 atributos

    public void procesarTodo() {
        // ... 100 líneas de lógica mezclada olor a método largo
    }
}
```

2. Nombres (Nomenclatura Pobre)

El código debe ser autodescriptivo, si necesitas leer el cuerpo de una función para entender qué hace porque el nombre es vago, hay un olor nombres ambiguos uso de letras sueltas a, b, x o términos genéricos data, info, manager nombres inconsistentes mezclar idiomas o usar diferentes verbos para la misma acción ej. getUsers, fetchClients, retrieveCustomers en la misma clase.

```
Java
# mal que es 'd' y que hace 'proc'?
def proc(d):
    for i in d:
        print(i)

# Bien:
def imprimir_lista_productos(lista_productos):
    for producto in lista_productos:
        print(producto)
```

3. Complejidad Innecesaria

Ocurre cuando el diseño es más complicado de lo que el problema requiere.

Generalización especulativa crear interfaces o jerarquías de clases por si acaso las necesitamos en el futuro, pero que hoy no hacen nada y clase de datos clases que solo tienen campos y getters/setters, sin comportamiento. A veces es mejor que esa lógica esté dentro de la clase en lugar de esparcida por fuera.

4. Código Duplicado

Es considerado el peor de los olores. Si cambias algo en un lugar, debes recordarlo en todos los demás, duplicación literal el mismo bloque de código en dos métodos de la misma clase y duplicación sutil dos métodos que hacen lo mismo pero con algoritmos ligeramente distintos.

```
Java
// logica de validación repetida
function crearPost(titulo) {
    if (titulo.length < 5) throw new Error("Muy corto");
    // guardar...
}

function actualizarPost(titulo) {
    if (titulo.length < 5) throw new Error("Muy corto");
    // actualizar...
}
```

5. Lógica Condicional

El uso excesivo de if-else o switch suele indicar que no se está aprovechando el polimorfismo. Switch statements, si tienes el mismo switch repetido en varias partes del código para decidir qué hacer según un tipo de objeto, deberías usar subclases Y cláusulas anidadas o código flecha, muchos if uno dentro de otro que empujan el código hacia la derecha.

```
Java
// Olor: cada vez que agregues un tipo, debes tocar este switch
public double calcularBono(Empleado e) {
    switch(e.tipo) {
        case "Gerente": return 1000;
        case "Programador": return 500;
        default: return 0;
    }
}

// Solución: cada clase empleado sabe calcular su propio bono.
```

Segundo. Investigación de olores entre clases:

1. Olores de Datos

Grupos de datos, ocurre cuando los mismos 3 o 4 parámetros aparecen juntos en múltiples clases o métodos ej. calle, ciudad, código postal esto indica que esos datos deberían ser una clase propia y objeto de datos, clases que solo tienen atributos y métodos de acceso. Son contenedores pasivos que obligan a otras clases a manipular sus entrañas, rompiendo el encapsulamiento.

2. Olores de Herencia

Legado rechazado una subclase hereda de una superclase pero solo usa una pequeña parte de sus métodos. El resto le estorba o no tienen sentido ejemplo una clase patodegoma que hereda de ave, pero tiene que lanzar una excepción en el método volar(). También jerarquías de herencia paralelas, cada vez que creas una clase en una jerarquía ej. modelo, te ves obligado a crear otra clase en otra jerarquía ej. controlador.

3. Olores de Responsabilidad y Acoplamiento

Envidia de atributos un método de la clase A parece estar más interesado en los datos de la clase B que en los suyos propios, intimidación inapropiada, dos clases están demasiado ligadas y conocen detalles privados implementación la una de la cual. Esto hace que sea imposible cambiar una sin romper la otra. Y clase intermediaria una clase cuya única función es delegar el trabajo a otra. Si una clase solo dice "pregúntale a él", quizá no debería existir.

4. Olores de Ajuste al Cambio

Estos son los más peligrosos para el mantenimiento a largo plazo: Cambio Divergente cuando tienes que modificar una misma clase por muchas razones diferentes y cirugía de escopeta el opuesto al anterior, cada vez que quieres hacer un pequeño cambio, tienes que hacer 10 modificaciones pequeñas en 10 clases diferentes. Es fácil olvidar alguna y generar bugs.

5. Biblioteca de Clases Incompleta

Ocurre cuando una biblioteca externa que estás usando no provee un método necesario. En lugar de extender la biblioteca o crear un wrapper, los desarrolladores suelen meter esa lógica dentro de sus propias clases de negocio, "manchándolas" con detalles técnicos externos.

Impacto en la Arquitectura del Software

Estos olores no solo hacen que el código sea feo, sino que degradan la arquitectura de las siguientes formas fragilidad, un cambio en una parte del sistema rompe lugares inesperados. Rigidez, es muy difícil y costoso añadir nuevas funcionalidades porque todo está entrelazado. Inmovilidad, no puedes reutilizar una clase en otro proyecto porque viene pegada a otras 20 clases.

Métodos para Identificarlos

Algunos de los que encuentre:

Análisis Estático: Herramientas como SonarQube o Linter que detectan automáticamente acoplamiento y falta de cohesión.

Métricas de Acoplamiento: Medir el CBO (Coupling Between Objects). Un número alto indica Intimidad Inapropiada.

Code Reviews: La revisión por pares es la mejor forma de detectar Envidia de Atributos o Legado Rechazado.

Mapas de Calor de Cambios: Si los archivos A, B y C siempre se modifican juntos en el historial de Git, tienes un olor de Cirugía de Escopeta.

Tercero. Análisis comparativo:

Característica	Olores DENTRO de Clases	Olores ENTRE Clases
Enfoque	Lógica, legibilidad y limpieza del código.	Estructura, jerarquía y comunicación.
Visibilidad	Se detectan leyendo el archivo fuente individual.	Se detectan mirando el diagrama de clases o el flujo de datos.
Principal Violación	Responsabilidad Única	Bajo Acoplamiento y Alta Cohesión.
Refactorización	Suele ser local	Requiere cambios estructurales

Afectación al mantenimiento

Olores Internos: Degradan la comprensión. Un método largo o nombres ambiguos obligan al desarrollador a gastar horas tratando de entender qué hace el código antes de poder arreglar un bug. Esto aumenta el costo de mantenimiento preventivo y correctivo.

Olores Entre Clases: Degradan la predictibilidad. La cirugía de escopeta hace que un cambio simple tenga efectos colaterales en archivos que no parecen estar relacionados. El mantenimiento se vuelve peligroso, ya que corregir un error suele introducir dos nuevos en otros módulos.

Afectación a la escalabilidad del software

Olores Internos: Generan deuda técnica acumulativa. Si no se eliminan los olores medibles como clases de 2000 líneas, llegará un punto en que la clase sea tan compleja que nadie se atreva a tocarla. Esto detiene la evolución de esa funcionalidad específica.

Olores Entre Clases: Destruyen la modularidad. Olores como la intimidad inapropiada o el cambio divergente crean un sistema monolítico y rígido.

Cuarto. Estudio de caso práctico:

Analice el código abierto del siguiente proyecto: <https://github.com/faker-js/faker>

Faker genera datos masivos de prueba (nombres, direcciones, transacciones financieras, etc.).

Identificación de Olores:

Hallazgos en el Faker.js, Al ser una biblioteca que debe proveer miles de métodos distintos, se enfrenta a problemas de organización de módulos.

A. Olores Dentro de Clases

Clase Grande, aunque Faker se ha modularizado, la clase principal que coordina todos los módulos tiende a crecer demasiado. Debe conocer a cada uno de los sub-módulos (animal, color, commerce, finance, etc.), acumulando cientos de propiedades.

Código Duplicado: Muchos módulos de localización repiten estructuras de datos idénticas. Aunque los valores cambian, la lógica de cómo se accede a un formato de dirección suele estar duplicada en varios archivos JSON o clases de utilidad de idiomas.

B. Olores Entre Clases

Envidia de Atributos, se observa en los módulos de helper. Algunos métodos de utilidad pasan más tiempo manipulando los datos internos de los objetos metadata o locales que realizando lógica propia.

Cirugía de escopeta si se decide cambiar la forma en que se estructuran las probabilidades lógica de aleatoriedad, actualmente hay que modificar casi todos los módulos individuales, ya que cada uno implementa su propia llamada al motor de azar (faker.number, faker.helpers.arrayElement).

Propuesta de Refactorización

Mi estrategia es el patrón estrategia y extracción de clase, para solucionar la cirugía de escopeta y la lógica condicional, propondría desacoplar la generación de la transacción de la lógica de negocio del tipo de transacción.

Código Refactorizado:

```
None
// solucion a logica condicional: usar una interfaz para tipos de transacciones
interface
TransactionStrategy {
  process(amount: number): TransactionResult;
```

```

}

class DepositStrategy implements TransactionStrategy {
  process(amount: number) {
    return { id: '123', amount, status: 'complete' };
  }
}

class WithdrawalStrategy implements TransactionStrategy {
  process(amount: number) {
    const status = amount > 500 ? 'pending_approval' : 'complete';
    return { id: '124', amount, status };
  }
}

// 2. Solución a Clase Grande y Duplicación: Inyectar utilidades de redondeo
class FinanceModule {
  private roundingHelper = new RoundingHelper(); // Clase extraída

  generateTransaction(strategy: TransactionStrategy) {
    const rawAmount = Math.random() * 1000;
    const cleanAmount = this.roundingHelper.twoDecimals(rawAmount);

    return strategy.process(cleanAmount);
  }
}

```

Beneficios de la propuesta para el proyecto

Reducción de la complejidad ciclomática: Al eliminar los if-else, el código es más fácil de probar con unit tests. Solo pruebas cada estrategia por separado.

Escalabilidad: Si el creador quiere añadir un nuevo tipo de transacción, no necesita tocar la clase FinanceModule. Solo crea una nueva.

Facilidad de Mantenimiento: El olor a duplicación se elimina al mover el redondeo a una clase de utilidad única compartida.